

SeedHammer II — the taproot pathological wallet

The hardest wallet this constellation describes, round-tripped on the machine.

This is the four-tier degrading vault — the wallet `CORPUS.md` calls *the* pathological example — expressed as a **depth-3 taproot tree**:

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,{and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/48'/0'/0'/2'<0;1>/*,@1/48'/0'/1'/2'<0;1>/*,@2/48'/0'/2'/2'<0;1>/*))),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/48'/0'/3'/2'<0;1>/*,@4/48'/0'/0'/2'<0;1>/*,@5/48'/0'/1'/2'<0;1>/*))),{and_v(v:older(65535),multi_a(2,@6/48'/0'/2'/2'<0;1>/*,@7/48'/0'/3'/2'<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/48'/0'/0'/2'<0;1>/*,@9/48'/0'/1'/2'<0;1>/*,@10/48'/0'/2'/2'<0;1>/*))}}})
```

Two sha256 hashlocks. Both Bitcoin timelock flavours, absolute and relative. `multi_a` at thresholds 3, 2, 2 and 1. A **NUMS internal key**, so there is no key-path spend at all — every route to the coins goes through a script leaf. Eleven cosigners across **three masters**.

Until three commits ago this document could not exist. Every one of its four leaves is `and_v(v:...)` wrapping a timelock or a hashlock, and the device's tap-leaf reader named only `pk / multi_a / sortedmulti_a`. It returned *zero* leaves and the consent screen read "Complex policy — display only", while the host derived the same wallet's addresses without complaint. That was F-214; the fix replaced the vocabulary with an emitter, and this journey is the proof it worked.

The keys are BIP-39 test vectors. Never put funds behind them.

The template carries eleven per-key origins, written by hand; the eleven key files declare them independently. If the two disagreed, the card would name a path the key does not live at — and **nothing downstream would notice**, because addresses come from the xpub a card carries, never from the origin it declares. That is F-217's lesson, and this gate is the answer to it.

2 · The engraved artifact

And nothing would stop it.

An earlier draft of this page said md encode "refuses outright". Measured: it refuses the *keyed* form, where it can see one fingerprint bound to several different keys — but on this *keyless* template --path exits **0** and produces a perfectly good card with eleven wrong origins. The check needs a fingerprint *and* an xpub per slot, and a keyless template carries neither (F-221). **The card that gets engraved is the form the guard cannot see**, so leaving --path off here is a choice this journey makes, not one the tool enforces.

The engraved set is 4 md1 chunk(s).


```
0975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*)),{and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi_a(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*)),{and_v(v:older(65535),multi_a(2,@6/<0;1>/*,@7/<0;1>/*)),and_v(v:older(4255898),multi_a(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))}}})
```

n: 11

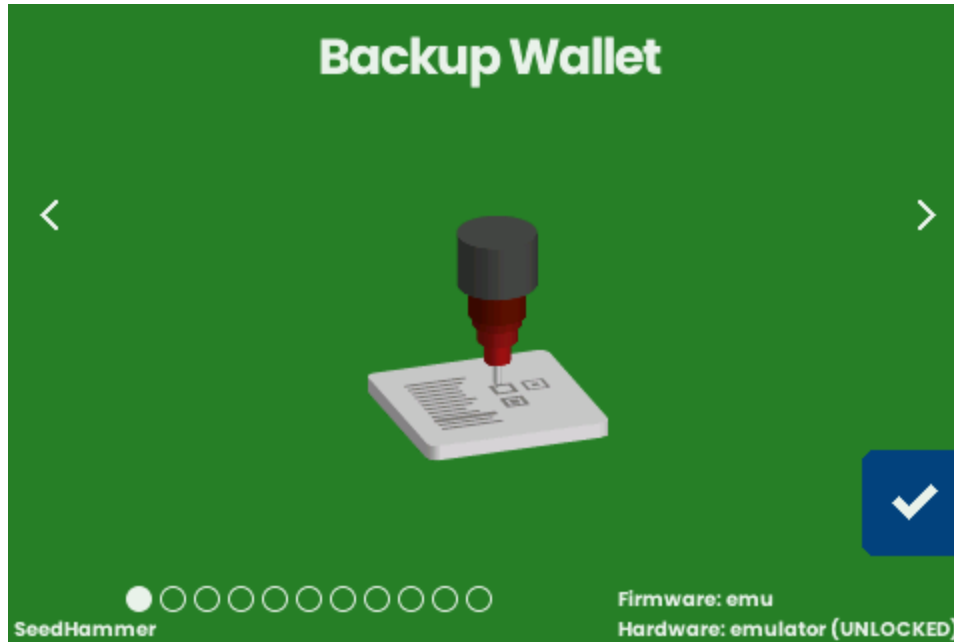
... clipped; full text in transcript_tr_pathological.txt

5 · What the device must prove to

Derived from the *cards*, not from the argument list that made them.

```
wallet idwallet-policy-id: 590f3abcaad2aca5a3f526917f5bb57a  
receive 0..1bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7pzz  
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnq1kny5nsgzguc0fcvd13s5hkkp5  
change 0..1bc1pnjrzcky5qq1hyz4rvhq5r96ca77wyccnfls0p517k8ae7402vjhshf3zya  
bc1p7yrpv9q6j58vvh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

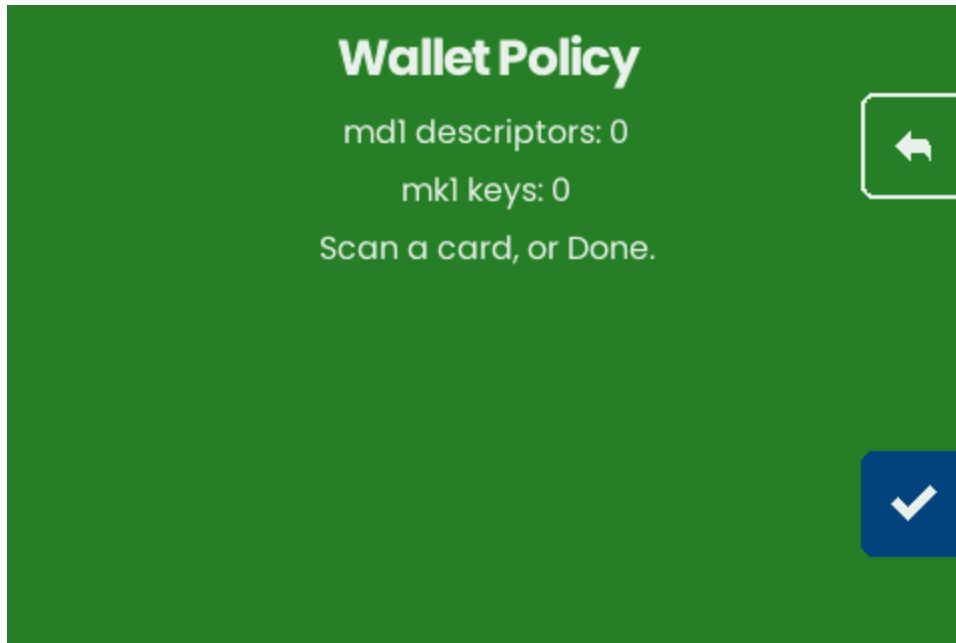
6 · The device



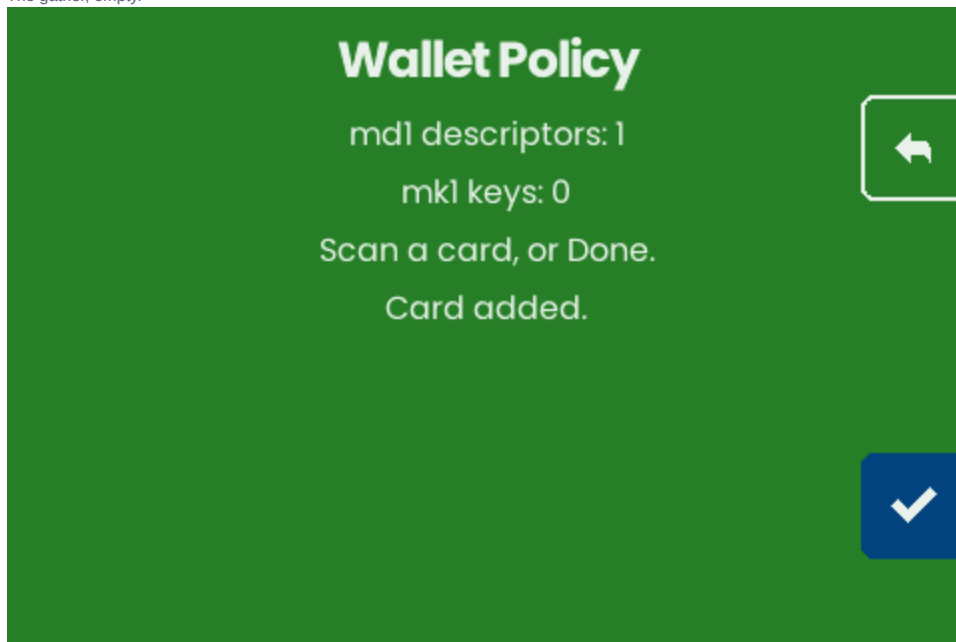
Home.



Wallet Policy.



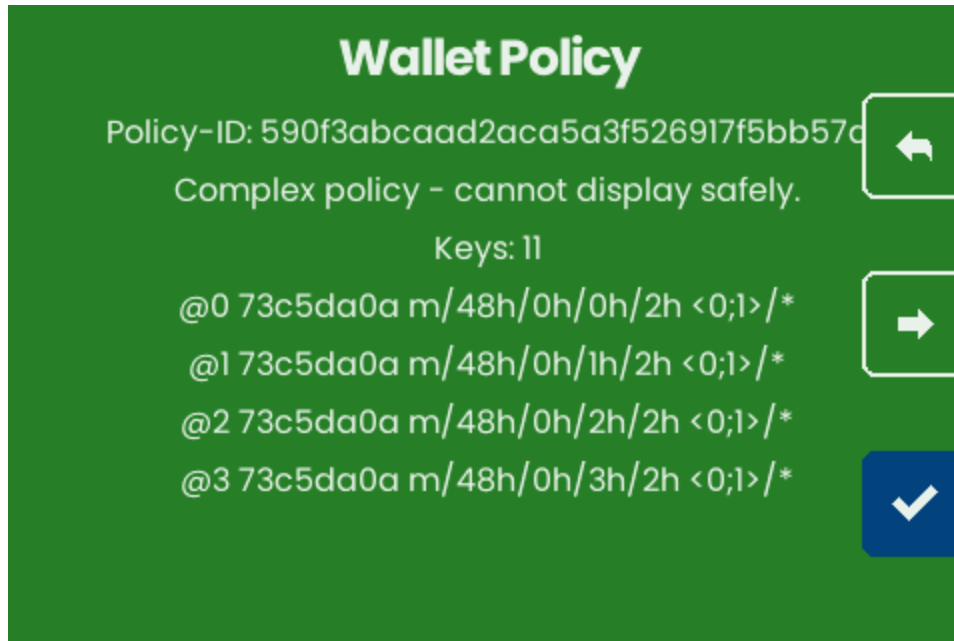
The gather, empty.



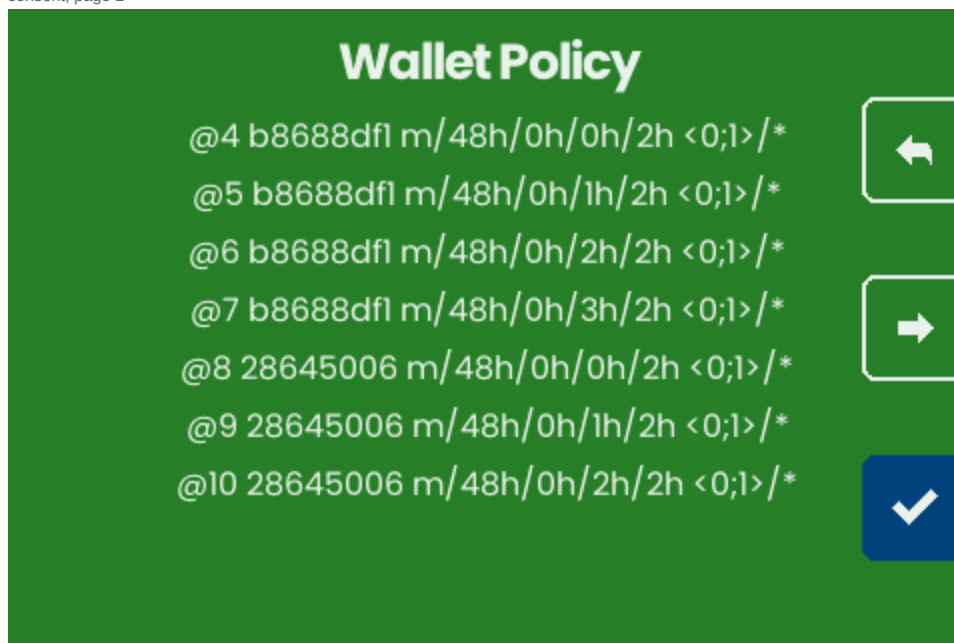
After all 24 chunks cross the reader: ONE card.

7 · The proof, on the consent path

Eleven key slots print before the addresses do, so the screen pages 4 times — and the walk reads every page, because a bound tuned for a four-key wallet would stop before reaching the half that matters.



consent, page 1



consent, page 2

Wallet Policy

Receive 0:
bc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7pzt

Receive 1:
bc1p3wy7tcypvkm49wtieq2yw4t0266vxdnqlkny5nsgzguc0fcvdl3s5hkkp5

Change 0:

←

→

✓

consent, page 3

Wallet Policy

bc1pnjrzcxy5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8advjhshf3zya

Change 1:
bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52xzs4h6khc

←

→

✓

consent, page 4

8 · What this run established

The capture is a **comparison**. It is handed the host's id and addresses and throws if the screen disagrees, and the absence assertions reject "display only" and "no addresses" outright — so the device cannot pass by refusing politely.

```
chunks presented24
cards gathered1
consent pages read4
wallet id matched590f3abcaad2aca5a3f526917f5bb57a
addresses matchedbc1pgavpejq7pkax6zdtuay9ejenangza2t3l6gfka79h94p23xcexssak7p2r
bc1p3wy7tcypvkm49wtyeq2yw4t0266vxdnqlkny5nsgzguc0fcvd13s5hkkp5
bc1pnjrzccky5qqlhyz4rvhq5r96ca77wyccnfls0p5l7k8ae7402vjhshf3zya
bc1p7yrpv9q6j58vwh5yrnnj8ky6dnykn7z2lak4j0ckt52j84j8fxzs4h6khc
```

The negative control is a command, not a memory. `python3 capture_tr_pathological.py --prove-it-can-fail` corrupts one character of one expected address and exits 0 only if the walk *caught* it. A comparison nobody has made fail is not evidence that it can.

Reproducing

```
bash transcript_tr_pathological.sh > transcript_tr_pathological.txt 2>&1
python3 capture_tr_pathological.py                # rebuilds emu.wasm, drives it
python3 capture_tr_pathological.py --prove-it-can-fail # the negative control
python3 build_pdf_tr_pathological.py                # this document
```

What this backup will NOT do

Tiers 1–2 cannot be spent from these plates alone. Both need the 32-byte preimage of the sha256 hashlock. **No plate carries it** — record it separately, with the vault, or those tiers are decoration.

The tiers do not open in the order they read. Tier 4 (1-of-3) matures at ~365 days, tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days first. Each `older()` clock runs per-coin from that coin's confirmation, not from the day you engraved.

In master terms this is a 1-of-3 vault with delays, not an 11-key multisig. Three of the four tiers need only one master: **A** alone for tier 1 (unlocked since 2016, but only with the secret word), **C** alone for tier 4 after ~365 days with no word, and **B** alone for tier 3 after ~455. Only tier 2 needs two, and it is the furthest out. The vault survives losing any two seed plates and falls to the theft of any one, given time.

What this journey does NOT show

- **No spend, and for tiers 1–2 no spend is possible from this backup at all.** Both hashlock tiers need the 32-byte preimage to spend, and *zero* backup strings carry it (F-132).
- **The tiers do not degrade in the order they read.** Tier 4 (1-of-3) matures at 365 days; tier 3 (2-of-2) at ~455 — the weakest key-set unlocks ~90 days *first* (F-133).
- **No plate read-back.** There is no reader; the card-set equality above is the honest proxy.
- **No ms1 secret leg.** Taproot-independent, and already shown in the earlier journeys.
- **Decode's output still does not round-trip** (F-219). The card carries its origins and both `md inspect` and `md decode` now report them — an earlier draft of this line said neither did, which its own section 4 already disproved. What remains is that decode's *stdout* renders them away, so re-encoding what it prints yields a different card.